

Week 03: Dynamic Data Structures

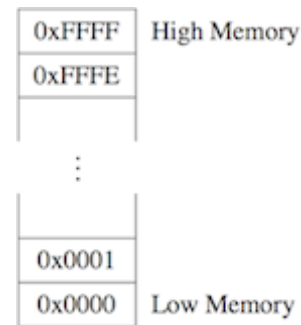
Memory

1/68

Reminder:

Computer memory ... large array of consecutive data cells or bytes

- `char` ... 1 byte
- `int, float` ... 4 bytes
- `double` ... 8 bytes
- `any_type *` ... 4 bytes (on CSE lab computers)



Memory addresses shown in Hexadecimal notation

C execution: Memory

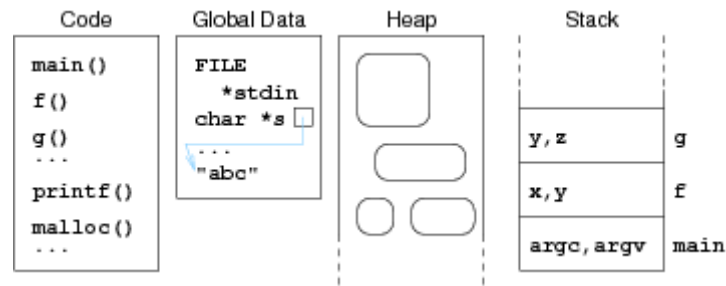
2/68

An executing C program partitions memory into:

- *code* ... fixed-size, read-only region
 - contains the machine code instructions for the program
- *global data* .. fixed-size, read-write region
 - contain global variables and constant strings
- *heap* ... very large, read-write region
 - contains dynamic data structures created by `malloc()` (see later)
- *stack* ... dynamically-allocated data (function local vars)
 - consists of frames, one for each currently active function
 - each frame contains local variables and house-keeping info

... C execution: Memory

3/68



Exercise #1: Memory Regions

4/68

```
int numbers[] = { 40, 20, 30 };

void insertionSort(int array[], int n) {
    int i, j;
    for (i = 1; i < n; i++) {
        int element = array[i];
        while (j >= 0 && array[j] > element) {
            array[j+1] = array[j];
            j--;
        }
        array[j+1] = element;
    }
}

int main(void) {
    insertionSort(numbers, 3);
    return 0;
}
```

Which memory region are the following objects located in?

1. `insertionSort()`
2. `numbers[0]`
3. `n`
4. `array[0]`
5. `element`

1. [code](#)
2. [global](#)
3. [stack](#)
4. [global](#)
5. [stack](#)

Dynamic Data Structures

Dynamic Memory Allocation

7/68

So far, we have considered *static* memory allocation

- all objects completely defined at compile-time
- sizes of all objects are known to compiler

Examples:

```
int    x;  
char  *cp;
```

```
typedef struct {float x; float y;} Point;  
Point p;  
char  s[20];
```

... Dynamic Memory Allocation

8/68

In many applications, fixed-size data is ok.

In many other applications, we need flexibility.

Examples:

```
char name[MAXNAME];  
char item[MAXITEMS];  
char dictionary[MAXWORDS][MAXWORDLENGTH];
```

With fixed-size data, we need to guess sizes ("large enough").

... Dynamic Memory Allocation

9/68

Fixed-size memory allocation:

- allocate as much space as we might ever possibly need

Dynamic memory allocation:

- allocate as much space as we actually need
- determine size based on inputs

But how to do this in C?

- all data allocation methods so far are "static"
 - however, stack data (when calling a function) is created dynamically (size is known)

Dynamic Data Example

10/68

Problem:

- read integer data from standard input (keyboard)
- first number tells how many numbers follow
- rest of numbers are read into a vector
- subsequent computation uses vector (e.g. sorts it)

Example input: 6 25 -1 999 42 -16 64

How to define the vector?

... Dynamic Data Example

11/68

Suggestion #1: allocate a large vector; use only part of it

```
#define MAXELEMS 1000
```

```
int numberOfElems;  
scanf("%d", &numberOfElems);
```

```
assert(numberOfElems <= MAXELEMS);
```

```
int i, vector[MAXELEMS];  
for (i = 0; i < numberOfElems; i++)  
    scanf("%d", &vector[i]);
```

Works ok, unless too many numbers; usually wastes space.

Recall that `assert()` terminates program with standard error message if test fails.

... Dynamic Data Example

12/68

Suggestion #2: use variables to give object sizes

```
int numberOfElems;  
scanf("%d", &numberOfElems);
```

```
int i, vector[numberOfElems];  
for (i = 0; i < numberOfElems; i++)  
    scanf("%d", &vector[i]);
```

Produces compiler error (compiler needs to know object sizes)

... Dynamic Data Example

13/68

Suggestion #3: create vector after count read in

```
#include <stdlib.h>
```

```
int numberOfElems;  
scanf("%d", &numberOfElems);
```

```
int i, *vector;  
size_t numberOfBytes;  
numberOfBytes = numberOfElems * sizeof(int);
```

```
vector = malloc(numberOfBytes);
assert(vector != NULL);

for (i = 0; i < numberOfElems; i++)
    scanf("%d", &vector[i]);
```

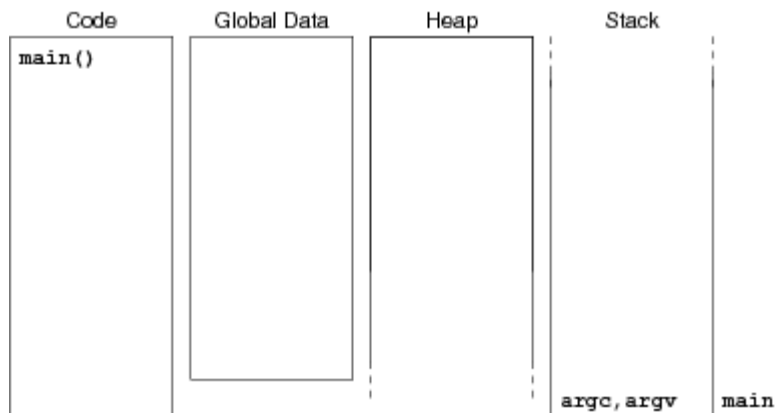
Works unless the *heap* is already full (very unlikely)

Reminder: because of pointer/array connection `&vector[i] == vector+i`

The `malloc()` function

14/68

Recall memory usage within C programs:



... The `malloc()` function

15/68

`malloc()` function interface

```
void *malloc(size_t n);
```

What the function does:

- attempts to reserve a block of `n` bytes in the *heap*
- returns the address of the start of this block
- if insufficient space left in the heap, returns `NULL`

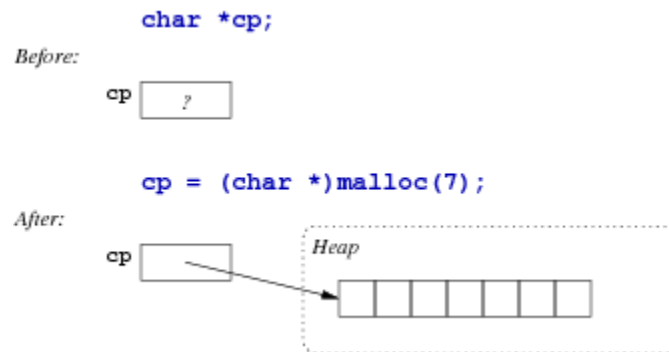
Note: `size_t` is essentially an unsigned int

- but has specialised interpretation of applying to memory sizes measured in bytes

... The `malloc()` function

16/68

Example use of `malloc`:



... The `malloc()` function

17/68

Things to note about `void *malloc(size_t):`

- it is defined as part of `stdlib.h`
- its parameter is a size in units of *bytes*
- its return value is a *generic* pointer (`void *`)
- the return value must *always* be checked (may be `NULL`)

Required size is determined by `#Elements * sizeof(ElementType)`

Exercise #2: Dynamic Memory Allocation

18/68

Write code to

1. create a dynamic $m \times n$ -matrix of floating point numbers, given m and n
2. create space for 1,000 speeding tickets (cf. Lecture Week 1)

How many bytes need to be reserved in each case?

1. Matrix:

```
2. float *matrix = malloc(m * n * sizeof(float));
3. assert(matrix != NULL);
```

$4 \cdot mn$ bytes allocated

4. Speeding tickets:

```
5. typedef struct {
6.     int day, month, year;
7. } DateT;
8. typedef struct {
9.     int hour, minute;
10. } TimeT;
11. typedef struct {
12.     char plate[7];
13.     DateT d;
14.     TimeT t;
15. } TicketT;
16.
17. TicketT *tickets = malloc(1000 * sizeof(TicketT));
18. assert(tickets != NULL);
```

28,000 bytes allocated

Exercise #3: Memory Regions

20/68

Which memory region is `tickets` located in? What about `*tickets`?

`tickets` is a variable located in the stack

`*tickets` is in the heap (after `malloc`'ing memory)

... The `malloc()` function

22/68

`malloc()` returns a pointer to a data object of some kind.

Things to note about objects allocated by `malloc()`:

- they exist until explicitly removed (program-controlled lifetime)
- they are *accessible* while some variable references them
- if no active variable references an object, it is *garbage*

The function `free()` releases objects allocated by `malloc()`

... The `malloc()` function

23/68

Usage of `malloc()` should always be guarded:

```
int *vector, length, i;
...
vector = malloc(length*sizeof(int));

assert(vector != NULL);

for (i = 0; i < length; i++) {
    ... vector[i] ...
}
```

Alternatively:

```
int *vector, length, i;
...
vector = malloc(length*sizeof(int));

if (vector == NULL) {
    fprintf(stderr, "Out of memory\n");
    exit(1);
}

for (i = 0; i < length; i++) {
    ... vector[i] ...
}
```

- `fprintf(stderr, ...)` outputs text to a stream called `stderr` (the screen, by default)
- `exit(v)` terminates the program with return value `v`

Memory Management

24/68

`void free(void *ptr)`

- releases a block of memory allocated by `malloc()`
- `*ptr` is a dynamically allocated object
- if `*ptr` was not `malloc()`'d, chaos will follow

Things to note:

- the contents of the memory block are not changed
- all pointers to the block still exist, but are not valid
- the memory may be re-used as soon as it is `free()`'d

... Memory Management

25/68

Warning! Warning! Warning! Warning!

Careless use of `malloc()` / `free()` / pointers

- can mess up the data in the heap
- so that later `malloc()` or `free()` cause run-time errors
- possibly well after the original error occurred

Such errors are **very difficult** to track down and debug.

Must be **very careful** with your use of `malloc()` / `free()` / pointers.

... Memory Management

26/68

If an uninitialised or otherwise invalid pointer is used, or an array is accessed with a negative or out-of-bounds index, one of a number of things might happen:

- program aborts immediately with a "segmentation fault"
- a mysterious failure much later in the execution of the program
- incorrect results, but no obvious failure

- correct results, but maybe not always, and maybe not when executed on another day, or another machine

The first is the most desirable, but cannot be relied on.

... Memory Management

27/68

Given a pointer variable:

- you can check whether its value is `NULL`
- you can (maybe) check that it is an address
- you **cannot** check whether it is a valid address

... Memory Management

28/68

Typical usage pattern for dynamically allocated objects:

```
Type *ptr = malloc(sizeof(Type));  
assert(ptr != NULL);
```

```
free(ptr);
```

```
int nelems = NumberOfElements;  
ElemType *arr = malloc(nelems*sizeof(ElemType));  
assert(arr != NULL);
```

```
free(arr);
```

Memory Leaks

29/68

Well-behaved programs do the following:

- allocate a new object via `malloc()`
- use the object for as long as needed
- `free()` the object when no longer needed

A program which does not `free()` each object before the last reference to it is lost contains a *memory leak*.

Such programs may eventually exhaust available heap space.

Exercise #4: Dynamic Arrays

30/68

Write a C-program that

- prompts the user to input a positive number n
- allocates memory for two n -dimensional floating point vectors **a** and **b**
- prompts the user to input $2n$ numbers to initialise these vectors
- computes and outputs the inner product of **a** and **b**
- frees the allocated memory

Sidetrack: Standard I/O Streams, Redirects

31/68

Standard file streams:

- `stdin` ... standard input, by default: keyboard
- `stdout` ... standard output, by default: screen
- `stderr` ... standard error, by default: screen
- `fprintf(stdout, ...)` has the same effect as `printf(...)`
- `fprintf(stderr, ...)` often used to print error messages

Executing a C program causes `main(...)` to be invoked

- with `stdin`, `stdout`, `stderr` already open for use
-

... Sidetrack: Standard I/O Streams, Redirects

32/68

The streams `stdin`, `stdout`, `stderr` can be *redirected*

- redirecting `stdin`
- `prompt$ myprog < input.data`
- redirecting `stdout`
- `prompt$ myprog > output.data`
- redirecting `stderr`
- `prompt$ myprog 2> error.data`

Abstract Data Types

Abstract Data Types

34/68

Reminder: An *abstract data type* is ...

- an approach to implementing data types
- separates *interface* from *implementation*
- users of the ADT see only the interface
- builders of the ADT provide an implementation

E.g. does a client want/need to know how a Stack is implemented?

- ADO = *abstract data object* (e.g. a single stack)
- ADT = *abstract data type* (e.g. stack data type)

... Abstract Data Types

35/68

ADT *interface* provides

- an *opaque* user-view of the data structure (e.g. `stack *`)
- function signatures (prototypes) for all operations
- semantics of operations (via documentation)

- a contract between ADT and its clients

ADT *implementation* gives

- concrete definition of the data structure
- function implementations for all operations
- ... including for *creation* and *destruction* of instances of the data structure

ADTs are important because ...

- facilitate decomposition of complex programs
- make implementation changes invisible to clients
- improve readability and structuring of software

Stack as ADT

36/68

Interface (in `stack.h`)

```
typedef struct StackRep *stack;
```

```
stack newStack();
```

```
void dropStack(stack);
```

```
int StackIsEmpty(stack);
```

```
void StackPush(stack, int);
```

```
int StackPop(stack);
```

ADT `stack` defined as a *pointer* to an unspecified struct

Static/Dynamic Sequences

37/68

Previously we have used an *array* to implement a stack

- fixed size collection of heterogeneous elements
- can be accessed via index or via "moving" pointer

The "fixed size" aspect is a potential problem:

- how big to make the (dynamic) array? (big ... just in case)
- what to do if it fills up?

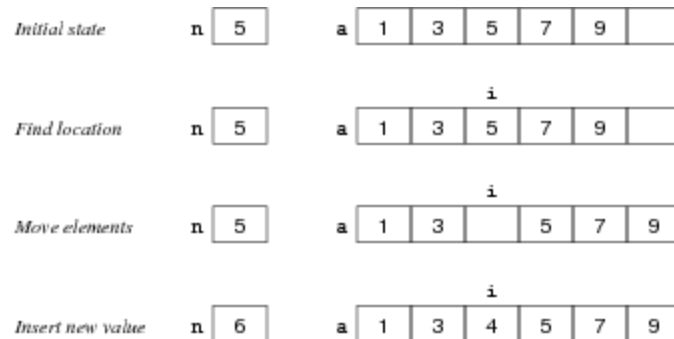
The rigid sequence is another problems:

- inserting/deleting an item in middle of array

... Static/Dynamic Sequences

38/68

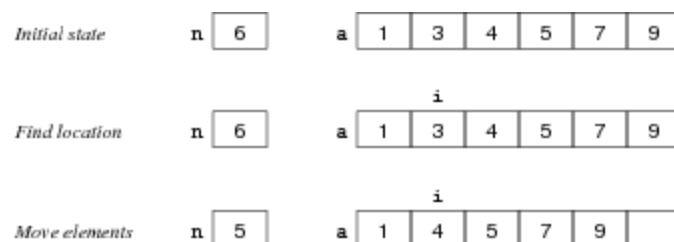
Inserting a value into a sorted array (`insert(a, &n, 4)`):



... Static/Dynamic Sequences

39/68

Deleting a value from a sorted array (`delete(a, &n, 3)`):

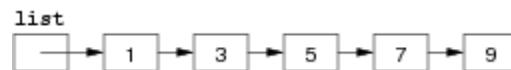


Dynamic Sequences

40/68

The problems with using arrays can be solved by

- allocating elements individually
- linking them together as a "chain"



Benefits:

- insertion/deletion have minimal effect on list overall
- only use as much space as needed for values

Self-referential Structures

41/68

To realise a "chain of elements", need a *node* containing

- a value
- a link to the next node

In C, we can define such nodes as:

```
struct node {  
    int data;  
    struct node *next;  
};
```

... Self-referential Structures

42/68

When defining self-referential types with `typedef`

```
typedef struct node {  
    int data;  
    struct node *next;  
} NodeT;
```

... Self-referential Structures

43/68

Note that the following definition does not work:

```
typedef struct {
    int data;
    NodeT *next;
} NodeT;
```

Because `NodeT` is not yet known (to the compiler) when we try to use it to define the type of the `next` field.

The following is also illegal in C:

```
struct node {
    int data;
    struct node recursive;
};
```

Because the size of the structure would have to satisfy `sizeof(struct node) = sizeof(int) + sizeof(struct node) = ∞`.

Linked Lists in C
